C QUICK REFERENCE GUIDE

Authorized Resource for CS210 — Version 23

USAF Academy Department of Computer and Cyber Science

Useful Header Files (add using the '#include <library>' directive)

```
stdio.h - input/output (printf, scanf)
math.h - math functions (use -lm to compile)
stdbool.h - defines bool data type
string.h - string comparison functions (see back)
```

```
1  #include <stdio.h>
2  #include "myfile.h"
```

Loading a standard library

Loading a library you wrote (must be in same folder)

gcc Compilation

Simple/Batch Compilation

Use this to compile your c files all at once.

```
gcc program.c -Wall -Werror -o program
```

- `program.c` — File(s) to compile
- `-Wall -Werror` — Error on warnings
- `-o program` — Output file (a.out if missing)

Separate Compilation

*Use this method when your program consists of multiple *.c files*

```
gcc program.c -Wall -Werror -c     produces program.o
gcc file1.c -Wall -Werror -c       produces file1.o
gcc program.o file1.o prog         links *.o files → prog
```

Main Program — Lesson 1

*Every C program has exactly one **main()** function. It is the first thing that runs when you start the program.*

```
1  #include <stdio.h>
2
3  // This main will return an int when done
4  int main() {
5
6    // Prints 'hello world' and a new line
7    printf("Hello World\n");
8
9    // Returns an integer (see line 4)
10   return 0;
11 }
```

printf() – print formatted, in the stdio.h library — Lessons 2/3

The printf() function lets you output text/values/variables to the screen.

```
printf("Welcome to CS210");                  // Text Only
>> Welcome to CS210

printf("%d %f", 5, x);                       // Values and Variables
>> 5 1.23                                       (x = 1.23)

printf("%d is my favorite number", 5);       // Text and values
>> 5 is my favorite number

printf("%08.5f", 1.234567890);               // Formatting Numbers
>> 01.23457                                  (8 chars, 5 dec, leading 0s)
```

`\n` = New Line `\t` = Tab `\\` = '\' character

Casting

```
float myValue = (float)1/(float)2;  // converts 1, 2 to floats
```

Format Specifiers — Lessons 2-4

See http://www.cplusplus.com/reference/cstdio/printf/

Data Type	character
int (32-bit single precision integer)	%d or %i
unsigned int	%u
long (64-bit double precision integer)	%li
float (32-bit single precision real #)	%f
double (64-bit double precision real #)	%lf
char (a single letter)	%c
scientific notation	%e
pointer address	p

scanf() – scan formatted; in the stdio.h library — Lesson 3

The scanf() function lets you get one or more values from the user. It does not print a prompt!

```
// Make sure you declare your variables before using scanf!
int    intVar = 0;
float  floatVar = 0.0;
double doubleVar = 0.0;
char   charVar = '';

// Various ways to get one or more values from the user
scanf("%d", &intVar);                  // int
scanf("%d%f", &intVar, &floatVar);     // int & float (ex: 1 2.34)
scanf("%c", &charVar);                 // 1 letter
scanf(" %c", &charVar);                // removes leading spaces
```

Variable Declaration

General Syntax

```
type var = value;
int numBunnies = 5;
float miles = 26.3;
```

Variables must start with letters, but can contain numbers and _. **Variable names are case sensitive**

Variable Assignment

Stored In

```
variable = exp/value;
x = 1;
var = a + b + 23;

// Shortcuts
x++;   // adds one
x+=2;  // adds two
```

.h and .c files — Lesson 5

.h files contain function declarations. This is the file you **#include** in your programs! **NOTE: a .h file typically has a matching .c**

myfuncts.h

```
#ifndef MYFUNCTS_H
#define MYFUNCTS_H

// Function declarations
int add(int num1, int num2);
int sub(int num1, int num2);

#endif
```

.c files contain definitions (i.e. code) for each .h function

myfuncts.c

```
// See how add() matches .h file
int add(int num1, int num2) {
    return num1 + num2;
}
```

Functions — Lesson 5

Functions let us define reusable chunks of code. Each function performs a single task.

```
float functionName(type parameter 1, type parameter2, ...) {
    // Your code goes here
    return value;
}
```

- Specifies what this function will return (void if nothing)
- Function Name
- Parameters (The Info the Function Needs)
- Return Value — This is what the function "gives back" to the caller
- Functions immediately end once they return a value

Calling Functions

Functions do not run by themselves. You must call them from other functions (i.e., main)

```
// Defining the function
float areaRectangle(float width, float height){
    return width * height;
}

// Calling the function
float w = 10.0
float h = 5.0
float area = areaRectangle(w, h);
```

The argument name does not have to match the parameter!

Conditional Logic — Lesson 7

```
if (LOGICAL TEST(S)) {
    DO WHEN ABOVE IS TRUE
}                                   } Required
else if (LOGICAL TEST(S)) {
    DO WHEN ABOVE IS TRUE           } Repeat as Needed
}
else {
    DO IF NOTHING IS TRUE           } Optional
}
```

Only the first TRUE block of code will execute.

Relational Operator	Symbol	Example
Equal To	==	name == 'bob'
Not Equal To	!=	x != 5
Greater Than	>	2023 > 2006
Greater Than or Equal To	>=	gpa >= 2.0
Less Than	<	21 < your_age
Less Than or Equal To	<=	x

Logical Operator	Description	Example
&&	True if BOTH conditions are True	GPA >= 3.0 && GPA <= 4.0
||	True if either condition is True	GPA < 2.0 || PEA < 2.0 || MPA < 2.0
!	True if the condition is not True	!(GPA < 2.0)

```
// Declaring/printing bool variables
bool isSunny = true;
printf("isSunny is %d", isSunny);
```

Characters / Strings — Lesson 9, 19

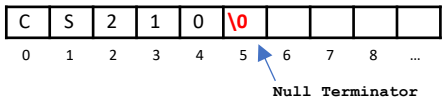
*A **character** is a single letter. It is denoted with single ticks*

```
// Declaring a single character (NOT a string)
char letter = 'u';
// Getting a char using scanf
scanf(" %c", &letter); & is required!
```

*A **string** is an array of characters. It is denoted w/double quotes*

```
// String of (max) 50 characters
char myString[50] = "CS210";
```

HOW myString IS REPRESENTED IN MEMORY




```
// Getting a single word (e.g., "hi")
char myString[50];
scanf("%s", myString); Do not put an &

// Getting multiple words (e.g., "hello world")
// Requires <string.h>
fgets(myString, 50, stdin);
                Max Size  Input
```

String Functions – Included with <string.h> — Lesson 9

```
// Declaring Variables
char string1[50] = "brown";
char string2[50] = "bear";
int result = 0;
```

```
// You cannot assign strings after declaration
string1 = "hello";  // not assignable

// Use strncpy instead!
strncpy(string1, "hello", 5); // works!
```

Due to security risks, it is best practice to use the green versions of each function.

```
// Gets the length of string1
result = strlen(string1);       // returns 5
result = strnlen(string1, 3);   // returns 3 since 3 is specified as max

// String Comparisons (returns ASCII difference of char that differs)
result = strcmp(string1, string1);  // returns 0 since they are equal
result = strcmp(string1, string2);  // returns 13 since r > e
result = strcmp(string2, string1);  // returns -13 since e > e
result = strncmp(string2, string1, 1); // returns 0 since b == b

// String Copying
strcpy(string1, string2); // copies string2 into string1 (adds '\0')
strncpy(string1, string2, 3); // copy first 3 chars of string2 to string1

// String Concatenation ("abc" + "def" = "abcdef")
strcat(string1, string2); // copies string2 to the end of string1
strncat(string1, string2, 3); // copies n chars of string2 to string1
```

Switch Statement — Lesson 11

***Switch statements** let a variable / value be tested for equality against a list of values.*

```
switch (variable) {
 case 'A':
   DO WHEN variable == 'A'
   break;  ← The break is needed to exit the switch
 case 'B':
   DO WHEN variable == 'B'
   break;
 default:  ← Do this if nothing above matches
   DO WHEN variable != A or B
}
```

Loops / Iteration — Lesson 12/13

For Loop

Repeats a predetermined number of times. Useful when you know in advance how many times the loop needs to execute.

Ex) Counting from 0-9, OR Repeating 10x

```
for (int i = 0; i < 10; i++) {
  // The code you want to execute
  // ten times goes here
}
```

Declare / Initialize Control Variable (i) — Exit when False — How to Modify i After Each Iteration

While (or Do-While) Loop

Continues executing the same sequence of code as long as the test condition evaluates to **True**.

Ex) Counting from 0-9

```
// Initialize the loop control variable
int i = 0;

// Test the loop control variable
while (x < 10) {
  # Code Goes Here
  i = i + 1;
}
```

While loop will run 0 or more times

```
// Initialize the loop control variable
int i = 0;

// Test the loop control variable
do {
  # Code Goes Here
  i = i + 1;
} while (x < 10);
```

Do-while loop will run 1 or more times

Pointers — Lesson 14

***Pointers** are variables that store memory addresses*

```
// Creating an integer pointer
int* p = NULL;
```

p ☐ *p is pointing at nothing*

```
// Pointing p at an integer
int value = 2024;
p = &value;
```

p ☐ → value 2024

p now contains value's address

```
// Modify the value stored at p
*p = 2006
```

p ☐ → value 2006

*p = "the value contained at the memory address"

```
// Printing p's address/value
printf("Addr:%p, value:%d, p:%d", p, value, *p);
>> Addr:0x<address>, value:2006, *p:2006
```

Use pointers to return multiple values from a function

```
// Defining the function
float swap(int* a, int* b){ // a and b are pointers
   int temp = *a;           // saves a's value
   *a = *b;                 // swaps a's value with b's
   *b = temp;               // sets b's value with a's
}
// Calling the function
int value1 = 10;          // Declaring 2 int variables
int value2 = 20;
swap(&value1, &value2);  // Passing the address of
                         // value1 and value2 to swap
```

Arrays — Lesson 15-16

An array is a set of indexed variables that share a common name.

One-Dimensional Arrays

```
int pft_scores[5] = {141,300,450,500,251};
```

This array is logically equivalent to:

index	0	1	2	3	4
values	141	300	450	500	251

```
// Create an Empty Array w/4 values
int my_array[4];

// Getting a Single Value by Index
my_array[0];

// Create an Array with Values
// Note the []
char another_array[] = {'a', 'i'};

// Prints Each Cell in the Array
for (int i = 0; i < 5; i++) {
  print("%d ", pft_scores[i];
}

// Function declaration w/array param
void my_function(int array[], int size);
```

No number needed — Size/length of array

Two-Dimensional Array

A two-dimensional array is basically a table

```
int myArray[3][2] = {{1,   123},
                     {2,   234},
                     {3,   345}};
```

Rows # Cols

***myArray** is logically equivalent to:*

	Column 0	Column 1
Row 0	1	123
Row 1	2	234
Row 2	3	345
Row 3	4	456

```
// Accessing a Specific Cell
myArray[row][column];

// Prints Each Row of the Array
for (int r=0; r < 4; r++){       // rows
   for (int c=0; c < 2; c++) {   // cols
      printf("%d ", myArray[r][c]);
   }
   printf("\n"); // done printing a row
}

// Function declaration w/2D array param
void my_function(int array[R][C]);
```

Optional (can be []) — Required

Enumerated Types — Lesson 17

***Enum** are user-defined types with user-defined values*

(C treats enum values as integers)

```
// Creating an enum for weekdays
// Unless specified, Mon=0, Tues=1, etc.
enum Weekday {Mon, Tues, Wed, Thur, Fri};

// Creating a variable of type Weekday
enum Weekday today = Mon;

// Printing an enum variable as an int
printf("%i", today);  // will print 0
```

fscanf & fprintf — Lesson 19

Lets you specify the source/destination for scanning/printing.

```
// Using fprintf
// stdout = Monitor
char str[100];
fprintf(stdout, "%s", str);

// Using fscanf
// stdin = Keyboard
fscanf(stdin, "%s", str);
```

Ctype – Included with <ctype.h> — Lesson 20

Lets you analyze characters and alter their values.

Ex) char c = 'a';

```
// Check if Letter
if (isalpha(c)) {
  // c is a letter
}

// Check if Number
if (isdigit(c)) {
  // c is 0-9
}

// Check if Upper Case
if (isupper(c)) {
  // c is upper case
}

// Check if Lower Case
if (islower(c)) {
  // c is lower case
}

// To Lower Case
c = tolower(c);

// To Upper Case
c = toupper(c);
```

File I/O (Green = Text File; Red = Binary File)

Lesson 21

```
// Create a File Pointer
FILE* file = NULL;
```

```
// Opening a Text File
file = fopen(<filename>, "<MODE>");
```

Mode	Meaning
r	Read existing file
w	Write to new file
a	Append to end of file
r+	Read/Write existing file
w+	Read/Write new file
rb	Read existing binary file
wb	Write new binary file
ab	Append to binary file
rb+	Read/Write existing binary
wb+	Read/Write new binary

```
// Do Something if Opening Failed
if (file == NULL) { // Do Something }
```

```
// If Needed:  Set Cursor Location
// type = int, float, etc
// offset = # of items to skip
// FLAG
//   SEEK_SET = "From Beginning"
//   SEEK_CUR = "From Current Location"
//   SEEK_END = "From End of File"
fseek(file,
      sizeof(<type>) * offset,
      FLAG);
```

READING

```
// Read File Contents
char line[100]; // for an entire line
char word[15];  // for a word

do {
  // Option 1:  Get an entire line
  fgets(line, 100, file);

  // Option 2:  Get individual value(s)
  fscanf(file, "%s", word);

} while (!feof(file));
```

```
// Read Contents
// dest = Pointer to variable/struct
// type = int, float, etc.
// # = number of values to read
fread(<dest>, sizeof(<type>), <#>, file);
```

WRITING

```
// Writing Text Contents
// Use normal printf formatting
// Values to Print
fprintf(file, "%s", value);
```

```
// Writing Binary Contents
// ptr = Pointer to variable/struct
// type = int, float, etc.
// # = number of values to write
fwrite(<ptr>, sizeof(<type>), <#>, file);
```

```
Close the file
fclose(file);
```

Structs

Lesson 23

Structs are a collection of variables. They are essentially a custom data type.

Struct Type Name (Formal)

```
typedef struct MyCadetInfo_struct {
   char name[50];
   int squad;
   int classYear;
} MyCadetInfo;
```

Struct Variables

Struct Type Name Shortcut

```
// Creating a New Cadet
MyCadetInfo cadet1;

// Use dots to access the vars inside
strcpy(cadet1.name, "Liam");
cadet1.squad = 19;
cadet1.classYear = 2024;
```

Dynamic Memory

Lessons 24-25

Lets you request memory from the OS.
Requires <stdlib.h>

```
// Step 1: Create a pointer
int* newVar = NULL;

// Step 2: Call malloc()
// Allocates memory for a new var/array
```

Size (in bytes)

```
newVar = (int*)malloc(sizeof(int)*num);
```

NOTES:
- Typecast to match your variable
- Typecast has one more * than sizeof()
- Num = # of values you want to store in your variable
  - If you are creating an array, num = array length

```
// Only If Needed: Call realloc()
// Resizes a memory block
```

Pointer to Resize | Size (in bytes)

```
newVar = (int*)realloc(newVar, newsize);

// Step 3:  free()
// Returns memory back to the OS
// Every malloc needs a free!
free(newVar);
```

Recursion

Lesson 30-32

*Recursion is when a function **calls itself**. It is useful when problems can be broken down into smaller pieces.*

Generic Format

Recursive functions consist of 1) base cases(s) and 2) recursive case(s)

```
// Ex: Summing up numbers from 1 to n
int summation(int n) {
  // Base Case: n == 0
  if (n == 0) {
    return 0;
  }

  // Recursive Case
  return n + summation(n-1);
}
```

← Base case(s) are the problems we know the answer to

This is the smaller problem

Tail Recursion

This is where we include the current answer in our function call

```
// Same function, but w/tail recursion
int tail_sum(int n, int sumSoFar) {
  // Base Case: n == 0
  if (n == 0) {
    return sumSoFar;
  }

  // (Tail) Recursive Case
  return tail_sum(n-1, n + sumSoFar);
}
```

← We return the accumulated answer

Update sumSoFar

Bit Operators

Lesson 33

X	Y	X & Y
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	X | Y
0	0	0
0	1	1
1	0	1
1	1	1

X	Y	X ^ Y
0	0	0
0	1	1
1	0	1
1	1	0

X	~X
0	1
1	0

Right Shift	0011 >> 2	0000
Left Shift	0011 << 2	1100

Binary Numbers

Lesson 33

```
// This is an integer. Hello integer ☺
int i = 12;

// i is stored as Binary
0000 0000 0000 0000 0000 0000 0000 1100
```

1 1 0 0

$(1*2^3)+(1*2^2)+(0*2^1)+(0*2^0) = 12$

```
// You can use 0b to assign i in binary
int i = 0b1100;

// You can use 0x to assign i in hex
int i = 0xC;
```

0 = 0	1 = 1	2 = 2	3 = 3
4 = 4	5 = 5	6 = 6	7 = 7
8 = 8	9 = 9	10 = A	11 = B
12 = C	13 = D	14 = E	15 = F

Binary Masking

Lesson 34

Using AND to extract a specific portion of a larger number

```
  1101 1111 0010 0100
& 0000 0000 1111 0000   ← MASK
  0000 0000 0010 0000
```

We put 1's where we want to extract. NOTE: We might want to shift the result to read the value.

Binary Flipping

Lesson 34

Using XOR to flip specific bits in a larger number

```
  1101 1111 0010 0100
^ 0000 1111 1111 0000   ← MASK
  1101 0000 1101 0100
```

We put 1's where we want to flip. Everything else stays the same!

Command-Line Arguments

Lesson 35

***Command Line Arguments** are values that are provided to a program when it is initially executed*

To Give CLAs to Your Program

Include arguments in the terminal AFTER your program name. Each word will be an argument (use quotes to group words into a single argument)

```
./a.out argument1 "argument 2"
```

To Access Command-Line Arguments

Modify your main function to include two parameters (below):

Argument(s) | Arguments (Stored as Strings)

```
int main(int argc, char* argv[]) {
  // Prints All CLAs
  // argv[0] is the program filename
  for (int i = 0; i < argc; i++) {
    printf("Arg: %s\n", argv[i]);
  }
  return 0;
}
```